

Algorithms

Lecture #22

Syed Hamed Raza

O/1 Knapsack Problem

0/1 Knapsack Problem

- To compute entries of V we will imply an inductive approach.
- As a basis, $V[0, j] = 0$ for $0 \leq j \leq W$ since if we have no items then we have no value.
- We consider two cases:

O/1 Knapsack Problem

0/1 Knapsack Problem

Leave object i :

- If we choose to not take object i , then the optimal value will come about by considering how to fill a knapsack of size j with the remaining objects $\{1, 2, \dots, i - 1\}$.
- This is just $V[i - 1, j]$.

O/1 Knapsack Problem

0/1 Knapsack Problem

Take object i :

- If we take object i , then we gain a value of v_i
- But we use up w_i of our capacity.
- With the remaining $j - w_i$ capacity in the knapsack, we can fill it in the best possible way with objects $\{1, 2, \dots, i - 1\}$.
- This is $v_i + V[i - 1, j - w_i]$.
- This is only possible if $w_i \leq j$.

O/1 Knapsack Problem

0/1 Knapsack Problem

recursive formulation

$$V[i, j] = -\infty \quad \text{if } j < 0$$

$$V[0, j] = 0 \quad \text{if } j \geq 0$$

$$V[i, j] =$$

$$\begin{cases} V[i - 1, j] & \text{if } w_i > j \\ \max\{ V[i - 1, j], v_i + V[i - 1, j - w_i] \} & \text{if } w_i \leq j \end{cases}$$

O/1 Knapsack Problem

O/1 Knapsack Problem

- A naive evaluation of this recursive definition is exponential.
- So, as usual, we avoid re-computation by making a table.
- Consider an example: max weight W is 11. There are five items to choose from

O/1 Knapsack Problem: DP

0/1 Knapsack Problem: DP

Weight limit (j):	0	1	2	3	4	5	6	7	8	9	10	11
$w_1 = 1 \ v_1 = 1$												
$w_2 = 2 \ v_2 = 6$												
$w_3 = 5 \ v_3 = 18$												
$w_4 = 6 \ v_4 = 22$												
$w_5 = 7 \ v_5 = 28$												

- The $[i, j]$ entry here will be $V[i, j]$, the best value obtainable using the first i rows of items if the maximum capacity were j .

O/1 Knapsack Problem: DP

0/1 Knapsack Problem: DP

Initialization and first row.

Weight limit (j):	0	1	2	3	4	5	6	7	8	9	10	11
$w_1 = 1 \ v_1 = 1$	0	1	1	1	1	1	1	1	1	1	1	1
$w_2 = 2 \ v_2 = 6$	0											
$w_3 = 5 \ v_3 = 18$	0											
$w_4 = 6 \ v_4 = 22$	0											
$w_5 = 7 \ v_5 = 28$	0											

Recall that we take $V[i, j]$ to be 0 if either i or j is ≤ 0 .

O/1 Knapsack Problem: DP

0/1 Knapsack Problem: DP

Fill in top-down, left-to-right.

Weight limit

(j):	0	1	2	3	4	5	6	7	8	9	10	11
$w_1 = 1 \ v_1 = 1$	0	1	1	1	1	1	1	1	1	1	1	1
$w_2 = 2 \ v_2 = 6$	0	1	6	7	7	7	7	7	7	7	7	7
$w_3 = 5 \ v_3 = 18$	0											
$w_4 = 6 \ v_4 = 22$	0											
$w_5 = 7 \ v_5 = 28$	0											

Always using

$$V[i, j] = \max\{V[i - 1, j], v_i + V[i - 1, j - w_i]\}$$

O/1 Knapsack Problem: DP

0/1 Knapsack Problem: DP

Weight limit (j):	0	1	2	3	4	5	6	7	8	9	10	11
$w_1 = 1 \ v_1 = 1$	0	1	1	1	1	1	1	1	1	1	1	1
$w_2 = 2 \ v_2 = 6$	0	1	6	7	7	7	7	7	7	7	7	7
$w_3 = 5 \ v_3 = 18$	0	1	6	7	7	18	19	24	25	25	25	25
$w_4 = 6 \ v_4 = 22$	0											
$w_5 = 7 \ v_5 = 28$	0											

$V[3, 7] = \max\{V[3 - 1, 7], v_3 + V[3 - 1, 7 - w_3]\}$
 $= \max\{V[2, 7], 18 + V[2, 7 - 5]\}$
 $= \max\{7, 18 + 6\} = 24$

O/1 Knapsack Problem: DP

0/1 Knapsack Problem: DP

Fill in top-down, left-to-right.

Weight limit

(j):	0	1	2	3	4	5	6	7	8	9	10	11
$w_1 = 1 \ v_1 = 1$	0	1	1	1	1	1	1	1	1	1	1	1
$w_2 = 2 \ v_2 = 6$	0	1	6	7	7	7	7	7	7	7	7	7
$w_3 = 5 \ v_3 = 18$	0	1	6	7	7	18	19	24	25	25	25	25
$w_4 = 6 \ v_4 = 22$	0	1	6	7	7	18	22	24	28	29	29	40
$w_5 = 7 \ v_5 = 28$	0											

Always using

$$V[i, j] = \max\{V[i - 1, j], v_i + V[i - 1, j - w_i]\}$$

O/1 Knapsack Problem: DP

0/1 Knapsack Problem: DP

Fill in top-down, left-to-right.

Weight limit (j):	0	1	2	3	4	5	6	7	8	9	10	11
$w_1 = 1 \ v_1 = 1$	0	1	1	1	1	1	1	1	1	1	1	1
$w_2 = 2 \ v_2 = 6$	0	1	6	7	7	7	7	7	7	7	7	7
$w_3 = 5 \ v_3 = 18$	0	1	6	7	7	18	19	24	25	25	25	25
$w_4 = 6 \ v_4 = 22$	0	1	6	7	7	18	22	24	28	29	29	40
$w_5 = 7 \ v_5 = 28$	0	1	6	7	7	18	22	28	29	34	35	40

Final answer: 40 (the bottom-right entry).

O/1 Knapsack: DP Algorithm

0/1 Knapsack: DP Algorithm

KNAPSACK(n, W)

1 for $w = 0, W$

2 do $V[0, w] \leftarrow 0$

3 for $i = 0, n$

4 do $V[i, 0] \leftarrow 0$

5 for $w = 0, W$

6 do if $(w_i \leq w \ \& \ v_i +$

$V[i - 1, w - w_i] > V[i - 1, w])$

7 then $V[i, w] \leftarrow v_i + V[i - 1, w - w_i]$

8 else $V[i, w] \leftarrow V[i - 1, w]$

Time complexity: clearly $O(n \cdot W)$

O/1 Knapsack Problem: DP

O/1 Knapsack Problem: DP

Constructing the Optimal Solution

- The algorithm for computing $V[i, j]$ does not keep record of which subset of items gives the optimal solution.
- To compute the actual subset, we can add an auxiliary boolean array $keep[i, j]$ which is 1 if we decide to take the i th item and 0 otherwise.

O/1 Knapsack Problem

O/1 Knapsack Problem

Question: How do we use all the values $keep[i, j]$ to determine the optimal subset T of items?

- If $keep[n, W]$ is 1, then $n \in T$. We can now repeat this argument for $keep[n - 1, W - w_n]$.
- If $kee[n, W]$ is 0, the $n \notin T$ and we repeat the argument for $keep[n - 1, W]$.

0/1 Knapsack Problem: DP

0/1 Knapsack Problem: DP

Weight limit (j):	0	1	2	3	4	5	6	7	8	9	10	11
$w_1 = 1 \ v_1 = 1$	0	1	1	1	1	1	1	1	1	1	1	1
$w_2 = 2 \ v_2 = 6$	0	0	1	1	1	1	1	1	1	1	1	1
$w_3 = 5 \ v_3 = 18$	0	0	0	0	0	1	1	1	1	1	1	1
$w_4 = 6 \ v_4 = 22$	0	0	0	0	0	0	1	0	1	1	1	1
$w_5 = 7 \ v_5 = 28$	0	0	0	0	0	0	0	1	1	1	1	0

- Selected items: 4 3

Design and Analysis of Algorithms

O/1 Knapsack

O/1 Knapsack Problem

O/1 Knapsack Problem

KNAPSACK(n, W)

```
1  for  $w = 0, W$ 
2  do  $V[0, w] \leftarrow 0$ 
3  for  $i = 0, n$ 
4  do  $V[i, 0] \leftarrow 0$ 
5      for  $w = 0, W$ 
6      do if ( $w_i \leq w$  &  $v_i +$ 
            $V[i - 1, w - w_i] > V[i - 1, w]$ )
```

O/1 Knapsack Problem

O/1 Knapsack Problem

Therefore, the following partial program will output the elements of T: . . . ()

```
1  k ← W
2  for i ← n downto 1
3  do if (keep[i, k] = 1)
4      then output i
5      k ← k - wi
```